# UML PlantUML dla serwera EKG

https://gemini.google.com/app/27aba887eef7145f

*User prompt: Chcę stworzyć opis w UML do opisania tego co potrzebuję stworzyć do pełnego działania tego serwera. Na pewno potrzebuję diagram klas, diagram komponentów, diagram sekwencji (dla pojedynczego pomiaru, nawiązania i zamknięcia połączenia). Daj mi te diagramy jako gotowy kod do plantuml. Cały mechanizm ma się składać z serwera, urządzenia wysyłającego ekg (paczka 250 sygnałów co sekundę, razem z cachem i wstrzymaniem przez brak zasięgu) Chciałbym aby serwer weryfikował istnienie połączenia co sekundę i "był gotowy" na przywrócenie połaczenia i pobranie większej ilości danych. Istnienie urzadzenia jest poza zakresem tego projektu, ale nalezy zaprojektować odpowiedni format paczki danych który będzie odbierany, razem z opisem koniecznych pól. Zakłada się, że dane wysyłane są bezpośrednio do serwera z powodu isnitenia modułu w urzadzenia zdalnego do połączeń wss://*

## Response: Specyfikacja paczki danych EKG (WSS Payload)

Paczka przesyłana jest przez urządzenie w formacie JSON (lub binarnym) co sekundę. Zawiera bufor 250 próbek sygnału (próbkowanie 250 Hz) oraz metadane umożliwiające wykrycie luk w transmisji po utracie zasięgu.

```
{
  "device_id": "DEV-98765-ECG",
  "session_id": "SESS-20261027-001",
  "sequence_number": 1042,
  "timestamp": 1793096880000,
  "sample_rate": 250,
  "samples_count": 250,
  "is_buffered": false,
  "samples": [0.012, 0.015, 0.025, 0.110, 0.850, 0.210, -0.050, ...]
}
```

### Opis pól struktury

| Pole | Typ | Opis |
|---|---|---|
| `device_id` | `string` | Unikalny identyfikator sprzętowy urządzenia (np. UUID lub numer seryjny). |
| `session_id` | `string` | Identyfikator ciągłej sesji pomiarowej. |
| `sequence_number` | `uint64` | Monotonicznie rosnący numer paczki. Służy do wykrywania zgubionych ramek i porządkowania danych buforowanych. |
| `timestamp` | `int64` | Czas wygenerowania pierwszej próbki w paczce (Unix Timestamp w milisekundach). |
| `sample_rate` | `int` | Częstotliwość próbkowania w Hz (domyślnie `250`). |
| `samples_count` | `int` | Liczba próbek w tablicy (domyślnie `250`). |

| Pole | Typ | Opis |
| --- | --- | --- |
| `is_buffered` | `bool` | Flaga określająca, czy paczka pochodzi z pamięci podręcznej (np. po wznowieniu zasięgu). |
| `samples` | `[]float64` | Tablica wartości napięcia sygnału EKG (w mV lub µV). |

# 1. Diagram klas (Class Diagram)

Struktura uwzględnia podział na pakiety `internal/websocket`, `internal/ecg` oraz `internal/storage` wraz z mechanizmem kontroli połączenia (Heartbeat/Ticker) oraz obsługą kodów zamknięcia (np. 1000, 3000).

```
@startuml
skinparam classAttributeIconSize 0

package "cmd/server" {
    class Server {
        - addr: string
        - upgrader: websocket.Upgrader
        + Start(): error
    }
}

package "internal/websocket" {
    class Handler {
        - upgrader: websocket.Upgrader
        - connManager: *ConnectionManager
        - ecgService: *ecg.Service
        + HandleConnection(w: http.ResponseWriter, r: *http.Request)
    }

    class ConnectionManager {
        - sessions: map[string]*ClientSession
        - heartbeatTicker: time.Ticker
        + Register(session: *ClientSession)
        + Unregister(id: string)
        + StartHeartbeatMonitor()
    }

    class ClientSession {
        - id: string
        - conn: *websocket.Conn
        - lastHeartbeat: time.Time
        - isAlive: bool
        + SendPing(): error
        + SendAck(seqNumber: uint64): error
        + Close(code: int, text: string): error
    }
}

package "internal/ecg" {
```

```
    class Service {
        - repo: Repository
        - parser: Parser
        + ProcessPayload(raw: []byte): (*ECDPacket, error)
    }

    class Parser {
        + Unmarshal(data: []byte): (*ECDPacket, error)
    }

    class ECDPacket {
        + DeviceID: string
        + SessionID: string
        + SequenceNumber: uint64
        + Timestamp: int64
        + SampleRate: int
        + SamplesCount: int
        + IsBuffered: bool
        + Samples: []float64
    }
}

package "internal/storage" {
    interface Repository {
        + SavePacket(packet: *ecg.ECDPacket): error
    }

    class TimeSeriesRepository {
        - db: DBConnection
        + SavePacket(packet: *ecg.ECDPacket): error
    }
}

Server --> Handler
Handler --> ConnectionManager
Handler --> Service
ConnectionManager "1" *-- "many" ClientSession
Service --> Parser
Service --> Repository
TimeSeriesRepository ..|> Repository
Parser ..> ECDPacket : tworzy
@enduml
```

# 2. Diagram komponentów (Component Diagram)

Diagram przedstawia architekturę modułową serwera w Go oraz jego relację z zewnętrznym urządzeniem EKG łączącym się po protokole `wss://`.

```
@startuml
package "Urządzenie Zdalne (Out of Scope)" {
    [Moduł EKG & Cache] as Device
}
```

```
node "Serwer Go (EKG Server)" {
    package "cmd/server" {
        [Main Application] as Main
    }

    package "internal/websocket" {
        [WSS Upgrader & Handler] as WSHandler
        [Connection & Heartbeat Manager] as ConnMgr
    }

    package "internal/ecg" {
        [ECG Parser & Validator] as Parser
        [ECG Service] as ECGService
    }

    package "internal/storage" {
        [ECG Repository] as Repo
    }
}

database "Baza Danych (np. TimeSeries DB)" as DB

Device -down-> WSHandler : wss:// (paczka 250 próbek/s)
Main -> WSHandler : Inicjalizacja
WSHandler -> ConnMgr : Zarządzanie sesją / Ping 1s
WSHandler -> Parser : Surowe bajty
Parser -> ECGService : Odczytana struktura EKG
ECGService -> Repo : Zapis pomiaru
Repo -> DB : Persystencja danych
@enduml
```

# 3. Diagram sekwencji (Sequence Diagram)

Diagram obejmuje nawiązanie połączenia, weryfikację stanu połączenia co 1 sekundę (Ping/Pong), odbiór pomiaru, przyspieszone pobieranie danych zbuforowanych po awarii zasięgu oraz kulturalne rozłączenie (kod 1000 / 3000).

```
@startuml
autonumber
actor "Urządzenie EKG\n(Klient WSS)" as Device
participant "WebSocket Handler" as WS
participant "Connection Manager" as ConnMgr
participant "ECG Service" as Service
database "Baza Danych" as DB

== 1. Nawiązanie połączenia WSS ==
Device -> WS: WSS Handshake (GET /ws)
WS -> WS: Upgrade HTTP do WebSocket
WS --> Device: 101 Switching Protocols
WS -> ConnMgr: Rejestracja sesji (ClientSession)

== 2. Pojedynczy pomiar oraz weryfikacja połączenia (Pętla 1s) ==
loop Co 1 sekundę
```

```
    Device -> WS: Paczka EKG (250 próbek, seq=101, is_buffered=false)
    WS -> Service: ProcessPayload(payload)
    Service -> DB: Zapis próbek do bazy
    WS --> Device: ACK (seq=101)

    ConnMgr -> Device: Ping Frame (sprawdzenie żywotności)
    Device --> ConnMgr: Pong Frame
end

== 3. Utrata zasięgu, buforowanie i ponowne nawiązanie połączenia ==
note over Device
  Brak zasięgu GSM/Wi-Fi.
  Urządzenie zapisuje paczki w lokalnym buforze (cache).
end note

... Brak komunikacji przez N sekund ...

ConnMgr -> ConnMgr: Wyrycie braku odpowiedzi Pong (Timeout)
ConnMgr -> ConnMgr: Oznaczenie sesji jako nieaktywnej

Device -> WS: Ponowne nawiązanie połączenia WSS (Re-connect)
WS --> Device: Połączenie aktywne
WS -> ConnMgr: Aktualizacja sesji

loop Pobieranie bufora danych (Catch-up Mode)
    Device -> WS: Paczka EKG (seq=102, is_buffered=true)
    WS -> Service: ProcessPayload(payload)
    Service -> DB: Zapis zaległych danych
    WS --> Device: ACK (seq=102)

    Device -> WS: Paczka EKG (seq=103, is_buffered=true)
    WS -> Service: ProcessPayload(payload)
    Service -> DB: Zapis zaległych danych
    WS --> Device: ACK (seq=103)
end

== 4. Zamknięcie połączenia ==
alt Kulturalne zamknięcie przez użytkownika (Kod 3000)
    Device -> WS: Close Frame (Kod 3000 "Closed by user")
    WS -> ConnMgr: Wyrejestrowanie sesji
    WS --> Device: Confirm Close Frame
else Wyłączenie urządzenia (Kod 1000)
    Device -> WS: Close Frame (Kod 1000 "Normal Closure")
    WS -> ConnMgr: Wyrejestrowanie sesji
    WS --> Device: Confirm Close Frame
end
@enduml
```

---